

PostgreSQL Synchronous Replication

Table of Contents

1. [Learning Objectives](#)
 2. [How Synchronous Replication Works](#)
 3. [Architecture Diagrams](#)
 4. [synchronous_standby_names Configuration](#)
 5. [Synchronous Commit Levels](#)
 6. [Transaction Commit Behavior](#)
 7. [Performance Impact](#)
 8. [Availability Considerations](#)
 9. [Quorum Synchronous Replication](#)
 10. [Per-Transaction Control](#)
 11. [Monitoring](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Interview Questions](#)
 15. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Configure synchronous replication with different standby selection modes
 - Understand the five synchronous commit levels and when to use each
 - Quantify and mitigate the performance impact of synchronous replication
 - Implement quorum-based synchronous replication for better availability
 - Override synchronous commit at the session or transaction level
 - Monitor synchronous commit performance and standby health
-

How Synchronous Replication Works

In asynchronous replication, the primary commits without waiting for standbys. In synchronous replication, the primary **waits** for a designated standby to confirm it has received (and optionally applied) the WAL before reporting the transaction as committed to the client.

Commit Protocol

ASYNCHRONOUS COMMIT:

```
Client —COMMIT—> Primary: Writes WAL —ACK—> Client
                        |
                        └─WAL—> Standby (later, best effort)
```

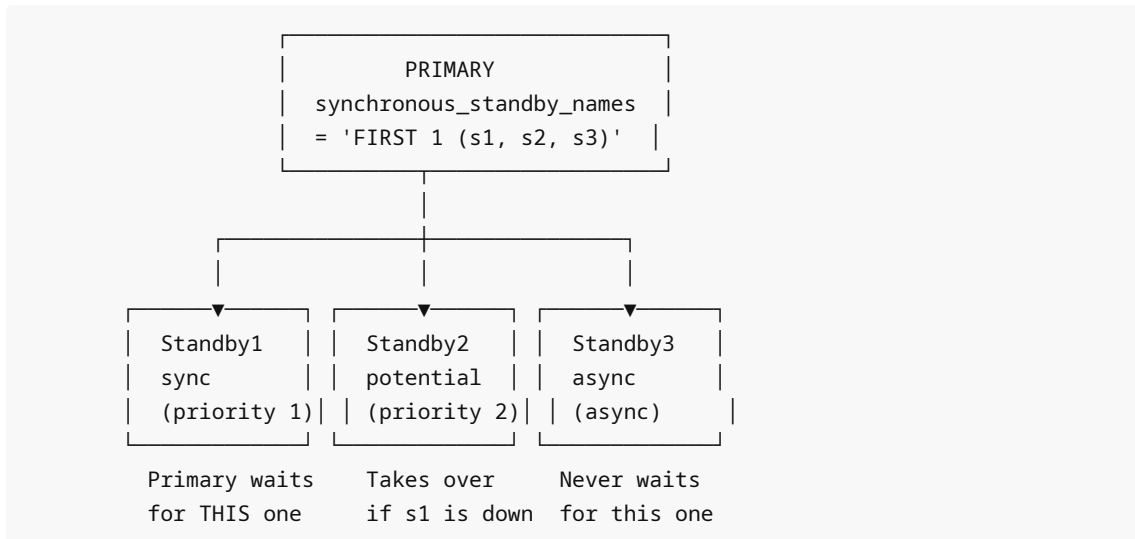
SYNCHRONOUS COMMIT:

```
Client —COMMIT—> Primary: Writes WAL —WAL—> Standby
                        |                               |
                        |                               └─ACK—> Primary
                        |
                        └─ACK—> Client (only after standby confirms)
```

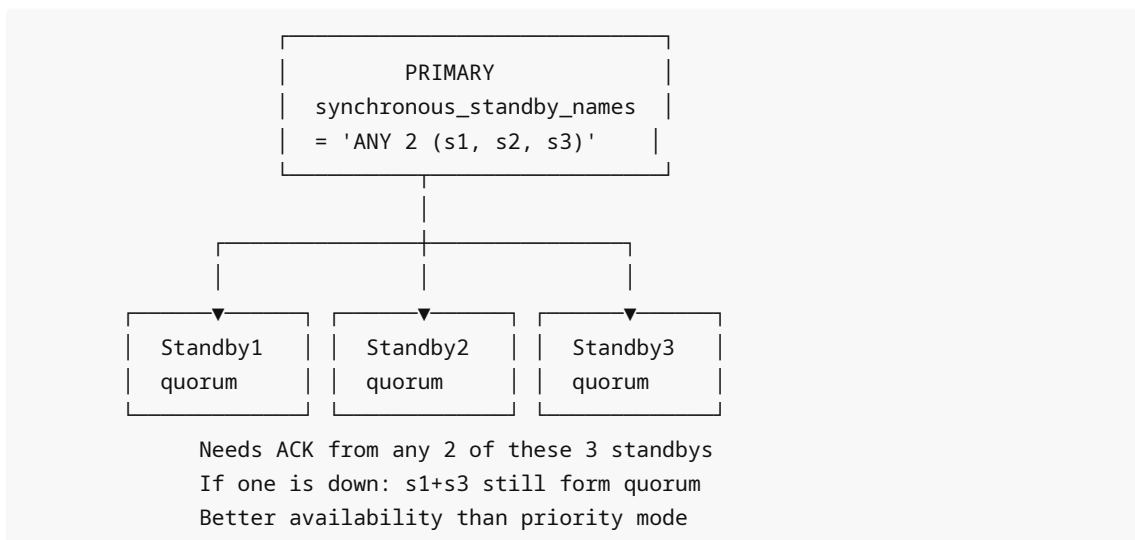
Result: If primary crashes after ACK, standby has the data.
RPO = 0 (zero data loss guarantee)

Architecture Diagrams

Priority-Based Synchronous



Quorum-Based Synchronous



synchronous_standby_names Configuration

Syntax Options

```
# postgresql.conf
```

```
# OPTION 1: Single standby (oldest syntax, still valid)
synchronous_standby_names = 'standby1'

# OPTION 2: FIRST N – Priority list (first N must confirm)
# Wait for the first 1 standby in priority order
synchronous_standby_names = 'FIRST 1 (standby1, standby2)'
# Wait for the first 2 standbys:
synchronous_standby_names = 'FIRST 2 (standby1, standby2, standby3)'

# OPTION 3: ANY N – Quorum commit (any N must confirm)
synchronous_standby_names = 'ANY 1 (standby1, standby2, standby3)'
synchronous_standby_names = 'ANY 2 (standby1, standby2, standby3)'

# OPTION 4: Empty – all standbys async (default)
synchronous_standby_names = ''
```

Application Name Matching

The `application_name` in the standby's `primary_conninfo` must match the name in `synchronous_standby_names`.

```
# On standby (postgresql.auto.conf):
primary_conninfo = 'host=primary port=5432 user=replicator
application_name=standby1'
```

```
-- Verify application names
SELECT application_name, sync_state, client_addr
FROM pg_stat_replication;

-- sync_state values:
-- 'sync'          : This standby is currently the synchronous one
-- 'potential'     : Would become sync if current sync standby fails
-- 'quorum'        : Part of quorum, but not necessarily all have to confirm
-- 'async'         : Not part of synchronous_standby_names
```

Dynamic Reconfiguration

```
-- Change synchronous standbys without restart (just reload)
ALTER SYSTEM SET synchronous_standby_names = 'ANY 2 (s1, s2, s3)';
SELECT pg_reload_conf();

-- Verify
SHOW synchronous_standby_names;
```

Synchronous Commit Levels

PostgreSQL offers five levels of synchronous commit, controlling exactly what the standby must confirm before the primary acknowledges commit.

synchronous_commit levels (from weakest to strongest guarantee):

off	Local WAL buffer flushed?	NO	Risk: lose up to 1 WAL buffer
local	Local WAL file flushed?	YES	Risk: lose if primary crashes
remote_write	Remote OS write?	YES	Risk: lose if standby host crashes
on	Remote WAL flush?	YES	Standard synchronous
remote_apply	Remote WAL applied?	YES	Strongest: standby reflects change

Configuration

```
# postgresql.conf – server-wide default
synchronous_commit = on          # Default
# or
synchronous_commit = remote_apply # Strongest guarantee

# Per-database:
ALTER DATABASE mydb SET synchronous_commit = 'remote_apply';

# Per-role:
ALTER ROLE financial_app SET synchronous_commit = 'on';

# Per-session:
SET synchronous_commit = 'local';  -- Only wait for local flush

# Per-transaction:
BEGIN;
SET LOCAL synchronous_commit = 'off';
INSERT INTO audit_log VALUES (...); -- Not critical, go fast
COMMIT;
```

Level Details

LEVEL: off

- Primary does NOT wait for WAL to be flushed locally
- COMMIT returns immediately after writing to WAL buffer
- Risk: up to 3 * wal_writer_delay (usually ~200ms) of transactions lost on crash
- Use case: Bulk loading, non-critical logging where some loss is acceptable
- Performance: Fastest (no fsync wait)

LEVEL: local

- Primary waits for local WAL fsync (durable on primary disk)
- Does NOT wait for standby at all
- Essentially: async replication from standby's perspective
- Use case: When you want local durability but can tolerate replication lag

LEVEL: remote_write (only meaningful with sync standby configured)

- Waits for standby to write WAL to OS (not necessarily flushed to disk)
- Protects against primary failure (standby has data in memory/OS cache)
- Risk: Standby server crash could lose data that wasn't flushed to disk
- Use case: Reasonably safe, better performance than 'on'

LEVEL: on (default synchronous)

- Waits for standby to flush WAL to disk (fsync on standby)
- Truly durable on standby disk
- Standard zero-data-loss synchronous replication
- Use case: Financial transactions, compliance requirements

LEVEL: remote_apply

- Waits for standby to apply WAL to database (visible in queries)
- Strongest guarantee: reads on standby will see this transaction
- Use case: When standby is used for read-after-write reads
- Performance: Slowest (extra round trip for apply)

Transaction Commit Behavior

The Blocking Problem

If the synchronous standby goes offline, ALL transactions that use synchronous commit will **block indefinitely** waiting for the standby to return.

Timeline:

T0: Standby1 disconnects
T1: Client issues COMMIT
T2: Primary writes WAL, waits for standby1 ACK
T3: ... waiting ...
T4: ... waiting ... (COMMIT blocks client indefinitely)
T5: DBA must intervene

Interventions When Standby is Down

```
-- Option 1: Switch to async for the session while standby is down
SET synchronous_commit = 'local';
-- All transactions in this session become async

-- Option 2: Change globally (affects all new transactions)
ALTER SYSTEM SET synchronous_standby_names = '';
SELECT pg_reload_conf();
-- WARNING: This disables synchronous replication entirely!

-- Option 3: Cancel blocking transactions
SELECT pg_cancel_backend(pid)
FROM pg_stat_activity
WHERE wait_event = 'SyncRep';

-- View blocking transactions
SELECT pid, query_start, wait_event, state, query
FROM pg_stat_activity
WHERE wait_event = 'SyncRep';
```

Performance Impact

Latency Impact

Synchronous replication adds network round-trip time to every transaction commit.

Commit latency = Local WAL flush + Network RTT + Standby WAL flush
(+ apply time if remote_apply)

Example:

Local flush:1ms
Network RTT:2ms (same datacenter)
Standby flush:1ms
Total:4ms per commit vs 1ms async

For remote datacenter:
Network RTT:20ms+
Total:22ms+ per commit

Benchmarking the Impact

```
# Test commit rate with async
pgbench -h primary -U postgres -d testdb -T 60 -c 10 -P 5

# Change to synchronous
psql -c "ALTER SYSTEM SET synchronous_standby_names = 'standby1';"
psql -c "SELECT pg_reload_conf();"

# Test commit rate with sync
pgbench -h primary -U postgres -d testdb -T 60 -c 10 -P 5

# Compare TPS (transactions per second) – expect 30-70% reduction for OLTP
```

Throughput vs. Safety Trade-off

Configuration	RPO	Commit Latency	Availability
synchronous_commit = off	~200ms	Minimal	Best
synchronous_commit = local	~0*	Low (local I/O)	Best
synchronous_commit = remote_write	0	Low+RTT	Good
synchronous_commit = on	0	Medium (RTT+I/O)	Fair
synchronous_commit = remote_apply	0	High	Fair
ANY N quorum (N+1 standbys)	0	Medium	Better than FIRST

* RPO~0 means primary loss only, standby may be behind

Quorum Synchronous Replication

Quorum replication (ANY N) provides zero data loss with better availability than priority (FIRST N).

Comparison

PRIORITY (FIRST 1):

Primary waits for standby1. If standby1 down → blocks ALL writes

Upside: Clear ordering, simple

Downside: Single point of failure in sync path

QUORUM (ANY 2 of 3):

Primary waits for ANY 2 of {s1, s2, s3}

If s1 down: s2 + s3 still form quorum → no blocking

Upside: Tolerates N-quorum failures

Downside: Slightly more network overhead

Quorum Configuration

```
# Require any 2 of 3 standbys
synchronous_standby_names = 'ANY 2 (standby1, standby2, standby3)'
```



```
# For 5-node cluster, require majority (3 of 5)
synchronous_standby_names = 'ANY 3 (s1, s2, s3, s4, s5)'
```

```
-- With quorum, all listed standbys show sync_state = 'quorum'
SELECT application_name, sync_state FROM pg_stat_replication;
```

application_name	sync_state
standby1	quorum
standby2	quorum
standby3	quorum

Per-Transaction Control

```
-- High-value financial transaction: strongest synchronous
BEGIN;
SET LOCAL synchronous_commit = 'remote_apply';
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
COMMIT;
```



```
-- Background analytics insert: async OK
BEGIN;
SET LOCAL synchronous_commit = 'off';
INSERT INTO analytics_events SELECT * FROM staging_events;
COMMIT;
```



```
-- Audit log: local durability is fine
BEGIN;
SET LOCAL synchronous_commit = 'local';
```

```
INSERT INTO audit_log (user_id, action, ts) VALUES ($1, $2, now());
COMMIT;
```

Monitoring

```
-- View sync state of all standbys
SELECT
    application_name,
    client_addr,
    state,
    sync_state,
    sync_priority,
    replay_lag,
    write_lag,
    flush_lag
FROM pg_stat_replication
ORDER BY sync_priority;

-- Count currently blocking transactions waiting for sync
SELECT COUNT(*) AS blocking_sync_transactions
FROM pg_stat_activity
WHERE wait_event = 'SyncRep';

-- Track sync commit latency via pg_stat_statements
SELECT query, calls, mean_exec_time
FROM pg_stat_statements
WHERE query LIKE '%COMMIT%'
ORDER BY mean_exec_time DESC
LIMIT 10;

-- Alert: transactions waiting > 5 seconds for sync
SELECT pid, query_start, now() - query_start AS wait_time, query
FROM pg_stat_activity
WHERE wait_event = 'SyncRep'
    AND now() - query_start > INTERVAL '5 seconds';
```

Common Mistakes

1. **Not realizing sync standby failure blocks all writes** — primary becomes unavailable when all sync standbys are down
2. **Using FIRST 1 with only one standby** — that one standby is a single point of failure for write availability
3. **Setting synchronous_commit = off globally** — accidentally removes all durability guarantees
4. **Mismatch between application_name and synchronous_standby_names** — standby stays async silently
5. **Using remote_apply in high-latency environments** — commit latency spikes significantly
6. **Not using quorum (ANY N)** — missing the availability benefits of quorum mode
7. **Not monitoring SyncRep wait events** — unaware that commits are blocked

8. **Not having a procedure for when sync standby goes down** — prolonged outage during standby failure
-

Best Practices

1. Use **ANY N quorum** with more standbys than quorum count for high availability
 2. Set **synchronous_commit = on** as the default; override to **off** for non-critical transactions
 3. Always maintain at least **N+1 standbys** where N is your quorum count
 4. Test the **standby-down scenario** before production: verify blocking behavior and recovery
 5. Use **per-transaction overrides** for bulk loads rather than changing global setting
 6. Monitor **SyncRep wait events** continuously
 7. Consider **geo-distribution**: sync standbys should be close (low RTT) to minimize latency impact
 8. Document and **automate recovery** from sync standby failure (Patroni handles this)
-

Interview Questions

Q1: What is the difference between `synchronous_commit = on` and `synchronous_commit = remote_apply` ?

A: `on` waits for the sync standby to flush WAL to disk (durable on standby). `remote_apply` waits for the standby to actually apply the WAL to its data files, making changes visible in standby queries. `remote_apply` is stronger: reads on the standby will see the committed data. `on` is faster: the standby might not have applied the changes yet even though they're durable on disk.

Q2: If the synchronous standby goes offline, what happens to write transactions on the primary?

A: All transactions with `synchronous_commit` set to `on`, `remote_write`, or `remote_apply` will block indefinitely at the COMMIT stage, waiting for standby acknowledgment. The primary cannot declare those commits complete. Solutions: switch to **ANY N quorum** with enough standbys, or use Patroni to automatically handle this.

Q3: Explain `synchronous_standby_names = 'ANY 2 (s1, s2, s3)'` .

A: This is quorum commit. The primary waits for ANY 2 of the 3 listed standbys to confirm WAL receipt. If s1 is offline, s2 and s3 still form a quorum, and commits continue normally. This provides zero data loss (RPO=0) while tolerating one standby failure. The `FIRST 2` alternative would require s1 AND s2 specifically (s3 only helps if s1 or s2 is down).

Q4: What is `synchronous_commit = 'off'` and is it safe?

A: With `off`, the COMMIT returns immediately without waiting for WAL to be flushed locally. Changes are still written to WAL buffers and will be flushed by the WAL writer within `wal_writer_delay` (default 200ms). The risk is losing up to 200ms of committed transactions if the server crashes before flushing. It is NOT the same as not writing WAL — the transaction is still recoverable after a clean shutdown. Use for non-critical high-throughput inserts (logging, analytics staging).

Q5: How does synchronous replication interact with the WAL sender timeout?

A: If a sync standby disconnects, the WAL sender process is terminated after `wal_sender_timeout`. Once the sender is gone, the primary removes that standby from its sync list. If using `FIRST 1 (s1)` and s1 disconnects, the primary immediately starts blocking. If using `ANY 2 (s1,s2,s3)` and s1 disconnects, s2+s3 still satisfy quorum.

Q6: Can you mix synchronous and asynchronous standbys?

A: Yes. Standbys listed in `synchronous_standby_names` are synchronous (or quorum); all others are asynchronous. Example: `synchronous_standby_names = 'FIRST 1 (s1, s2)'` — s1 and s2 are sync candidates, any other connected standbys are async. You can have 1 sync standby for HA and 3 async read replicas.

Q7: What is `sync_priority` in `pg_stat_replication` ?

A: `sync_priority` shows the standby's position in the `synchronous_standby_names` list. Priority 1 is the first listed (highest priority for `FIRST N` mode). With `ANY N` quorum mode, all listed standbys have equal priority and `sync_priority` reflects list order but doesn't determine which must confirm.

Q8: How would you implement zero data loss with good write availability?

A: Use `ANY 2 (s1, s2, s3)` with three standbys in the same data center. Any two must confirm each commit. Tolerate one standby failure without blocking writes. For geographic distribution: place standbys in nearby data centers (< 5ms RTT) to keep commit latency acceptable. Use Patroni for automatic failover if primary fails.

Exercises and Solutions

Exercise 1: Synchronous Configuration Lab

Configure primary with quorum synchronous replication:

```
-- Set up quorum sync with 3 standbys
ALTER SYSTEM SET synchronous_standby_names = 'ANY 2 (replica1, replica2, replica3)';
SELECT pg_reload_conf();

-- Verify all three are quorum
SELECT application_name, sync_state, replay_lag
FROM pg_stat_replication;
-- Expected: all three show 'quorum'
```

Exercise 2: Per-Transaction Sync Level

```
-- Demonstrate different sync levels for different transaction types

-- Critical financial transfer
CREATE OR REPLACE FUNCTION transfer_funds(
    from_account INT, to_account INT, amount NUMERIC
) RETURNS void AS $$
BEGIN
    SET LOCAL synchronous_commit = 'remote_apply'; -- Strongest guarantee
    UPDATE accounts SET balance = balance - amount WHERE id = from_account;
    UPDATE accounts SET balance = balance + amount WHERE id = to_account;
    INSERT INTO transfer_log VALUES (from_account, to_account, amount, now());
END;
$$ LANGUAGE plpgsql;
```

```
-- Non-critical metric collection
CREATE OR REPLACE FUNCTION log_page_view(user_id INT, page TEXT)
RETURNS void AS $$
BEGIN
    SET LOCAL synchronous_commit = 'off'; -- Speed over durability
    INSERT INTO page_views VALUES (user_id, page, now());
END;
$$ LANGUAGE plpgsql;
```

Exercise 3: Blocking Detection Query

```
-- Monitor and alert on sync replication blocking
-- Run this as a background monitoring job

WITH blocking AS (
    SELECT
        pid,
        username,
        application_name,
        query_start,
        now() - query_start AS wait_duration,
        query
    FROM pg_stat_activity
    WHERE wait_event = 'SyncRep'
),
standby_status AS (
    SELECT
        string_agg(application_name || ':' || sync_state, ', ') AS standbys,
        COUNT(*) FILTER (WHERE sync_state = 'sync') AS active_sync_standbys
    FROM pg_stat_replication
)
SELECT
    b.pid,
    b.wait_duration,
    ss.standbys AS replication_status,
    ss.active_sync_standbys
FROM blocking b
CROSS JOIN standby_status ss
WHERE b.wait_duration > INTERVAL '5 seconds'
ORDER BY b.wait_duration DESC;
```

Cross-References

- [02 streaming replication.md](#) — Setting up streaming replication
- [05 failover procedures.md](#) — What to do when sync standby fails
- [06 patroni.md](#) — Automatic sync standby management
- [09 ha architecture patterns.md](#) — HA with synchronous replication